>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 4

# Arithmetic, Logical and Relational Operators

Computing with values: expressions, precedence, the Math class and boolean logic

```
> dotnet run
7 / 2       -> 3
7 / 2.0     -> 3.5
-7 % 3      -> -1
0.1 + 0.2 == 0.3
-> False
> _
```

**Asst. Prof. Hasan Oğuz, PhD**
Istanbul Okan University · Vocational School
2026–2027 Fall Semester

>_ LEARNING OBJECTIVES

# By the end of this week you can …

## Arithmetic

Use + − * / % and predict whether the result is int or double.

## Precedence

Evaluate an expression step by step using precedence and associativity.

## Assignment

Use += −= *= /= %= and ++/−− (prefix vs postfix) with trace tables.

## Math class

Apply Pow, Sqrt, Abs, Round, Floor, Ceiling, Max and Min correctly.

## Relational

Compare values with == != < > <= >=; compare doubles safely.

## Logical

Combine conditions with && || !, short-circuit and De Morgan.

# From storing values to computing with them

```
Week 3 — variables and types
int count = 7;                // whole number
double price = 12.5;          // real number
bool isOpen = true;           // true / false
string name = "Ayse";         // text
const double VatRate = 0.20; // constant

double total = count * price * (1 + VatRate);
```



## Week 3: store

Declare, initialise, read and convert values.





## Week 4: compute

Operators combine values into new values.



**Rule of the week:** the TYPES of the operands decide the TYPE of the result. `7 / 2` is int, `7 / 2.0` is double.

# Anatomy of an expression




**Evaluated first:** `price * qty`, then `+ shipping`, finally = stores the result.

## Operand

A value the operator works on: a literal (7), a variable (price) or a sub-expression (a + b).

## Operator

Unary: -x  !ok  ++i
Binary: a + b  a < b
Ternary: c ? a : b

## Expression

Anything that produces a value. Every expression has a type: int, double, bool, string …



PART 1

# Arithmetic Operators

+ − * / % · integer vs real division · remainder tricks · division by zero

# The five arithmetic operators

| Operator | Meaning | int example | Prints | double example | Prints |
|---|---|---|---|---|---|
| + | addition | `7 + 2` | 9 | `7.0 + 2` | 9 |
| - | subtraction | `7 - 2` | 5 | `7.0 - 2` | 5 |
| * | multiplication | `7 * 2` | 14 | `7.0 * 2` | 14 |
| / | division | `7 / 2` | 3 | `7.0 / 2` | 3.5 |
| % | remainder (mod) | `7 % 2` | 1 | `7.5 % 2` | 1.5 |

**Only / behaves differently:** `7 / 2` gives 3 but `7.0 / 2` gives `3.5`.

### int op int → int

Both operands int: the result is int, even for /.

### double wins

At least one double operand: the int is widened, result is double.

# Integer division vs real division

code/04_02_DivisionRules (excerpt)

```
Console.WriteLine("--- integer vs real division ---");
Console.WriteLine($"7 / 2           = {7 / 2}");
Console.WriteLine($"7 / 2.0         = {7 / 2.0}");
Console.WriteLine($"(double)7 / 2   = {(double)7 / 2}");
Console.WriteLine($"(double)(7 / 2) = {(double)(7 / 2)}");
Console.WriteLine($"-7 / 2          = {-7 / 2}   (truncated toward zero)");
```

Console

```
--- integer vs real division ---
7 / 2           = 3
7 / 2.0         = 3.5
(double)7 / 2   = 3.5
(double)(7 / 2) = 3
-7 / 2          = -3   (truncated toward zero)
```

**Truncation**
int / int drops the fraction: 7 / 2 → 3, −7 / 2 → −3 (toward zero).

**Fix: make one operand double**
7 / 2.0 or (double)7 / 2 → 3.5. Cast an operand, before dividing.

**Too late!**
(double)(7 / 2) → 3: the division already happened in int.

# The remainder operator % and its uses

## Long division of 17 by 5

$$17 = 3 \times 5 + 2$$

`17 / 5` → 3 (quotient)

`17 % 5` → 2 (remainder)

```
// always true for int a, b (b != 0)
// a == (a / b) * b + a % b
// 17 == 3 * 5 + 2
```

### Even or odd?

n % 2 == 0 → even. n % 2 is 0 or 1 (or −1 for negative n!).

### Digits

n % 10 → last digit; n / 10 → drop the last digit. 472 → 2 and 47.

### Wrap-around

(hour + 5) % 24 → the hour 5 hours later: (22 + 5) % 24 = 3.

# % with negative numbers: the sign follows the dividend

| a | b | a / b | a % b | check: (a/b)*b + a%b |
|---|---|---|---|---|
| 7 | 3 | 2 | 1 | 2*3 + 1 = 7 |
| -7 | 3 | -2 | -1 | (-2)*3 + (-1) = -7 |
| 7 | -3 | -2 | 1 | (-2)*(-3) + 1 = 7 |
| -7 | -3 | 2 | -1 | 2*(-3) + (-1) = -7 |

**Mathematics:**

−7 mod 3 = 2  (always 0 … m−1)

**C#:**

-7 % 3 = -1  (sign of the dividend)

**The fix**

```
// safe modulo: result is always 0 .. m-1
int x = -7, m = 3;
int r = ((x % m) + m) % m;   // (-1 + 3) % 3 = 2
```



**Used today in:** clock arithmetic (going back in time) and the T.C. Kimlik check digit d10, where 7·odd − even can be negative.

# Division by zero: int crashes, double does not

## int → exception (program stops)

```
int n = 0;
Console.WriteLine(10 / n);
```

```
Console
Unhandled exception.
System.DivideByZeroException:
Attempted to divide by zero.
```

`10 / 0` (literal) → compile error CS0020: "Division by constant zero".

## double → special values (no crash)

```
double zero = 0.0;
Console.WriteLine($" 1.0 / 0.0 = {1.0 / zero}");
Console.WriteLine($"-1.0 / 0.0 = {-1.0 / zero}");
Console.WriteLine($" 0.0 / 0.0 = {0.0 / zero}");
```

```
Console
 1.0 / 0.0 = Infinity
-1.0 / 0.0 = -Infinity
 0.0 / 0.0 = NaN
```

**NaN** = "Not a Number". Infinity and NaN spread silently through later calculations.

>_ WORKED EXAMPLE 1

# Arithmetic calculator: all five operators

code/04_01_ArithmeticCalculator/Program.cs

```
Console.Write("First integer  a = ");
int a = int.Parse(Console.ReadLine());
Console.Write("Second integer b = ");
int b = int.Parse(Console.ReadLine());

Console.WriteLine($"a + b = {a + b}");
Console.WriteLine($"a - b = {a - b}");
Console.WriteLine($"a * b = {a * b}");
Console.WriteLine($"a / b = {a / b}   (quotient)");
Console.WriteLine($"a % b = {a % b}   (remainder)");
Console.WriteLine($"(double)a / b = {(double)a / b:F4}");

// quotient * divisor + remainder always gives back a
Console.WriteLine($"check: {a / b} * {b} + {a % b} = {a / b * b + a % b}");
```

Console

```
First integer  a = 17
Second integer b = 5
a + b = 22
a - b = 12
a * b = 85
a / b = 3   (quotient)
a % b = 2   (remainder)
(double)a / b = 3.4000
check: 3 * 5 + 2 = 17
```

a = -17, b = 5

```
a % b = -2   (remainder)
(double)a / b = -3.4000
check: -3 * 5 + -2 = -17
```



**b = 0?** the program stops at a / b with a DivideByZeroException. Week 5: check first with if.

>_ WORKED EXAMPLE 2

# Digits of a three-digit number

code/04_03_DigitExtraction (excerpt)

```
Console.Write("Enter a three-digit number: ");
int n = int.Parse(Console.ReadLine());

int hundreds = n / 100;        // 472 / 100 = 4
int tens = n / 10 % 10;        // 472 / 10 = 47, 47 % 10 = 7
int ones = n % 10;             // 472 % 10 = 2

int digitSum = hundreds + tens + ones;
int reversed = ones * 100 + tens * 10 + hundreds;
```

**4** n / 100

**7** n / 10 % 10

**2** n % 10

hundreds · tens · ones

Pseudocode

```
START
  READ n
  h ← n div 100
  t ← (n div 10) mod 10
  o ← n mod 10
  PRINT h, t, o, h + t + o
END
```

Console

```
Enter a three-digit number: 472
hundreds = 4, tens = 7, ones = 2
digit sum = 13
reversed  = 274
```

>_ WORKED EXAMPLE 3

# Seconds to days, hours, minutes, seconds

code/04_04_SecondsToHms (excerpt)

```
int days = total / 86400;              // 86400 s in a day
int hours = total % 86400 / 3600;      // what is left after whole days
int minutes = total % 3600 / 60;
int seconds = total % 60;

Console.WriteLine($"{total} s = {days} d {hours} h {minutes} min {seconds} s");
Console.WriteLine($"clock format: {days}d {hours:D2}:{minutes:D2}:{seconds:D2}");
```

| Expression (total = 100000) | Value |
|---|---|
| total / 86400 | 1 |
| total % 86400 | 13600 |
| total % 86400 / 3600 | 3 |
| total % 3600 / 60 | 46 |
| total % 60 | 40 |

Console

```
Total seconds: 3725
3725 s = 0 d 1 h 2 min 5 s
```

Console

```
Total seconds: 100000
100000 s = 1 d 3 h 46 min 40 s
```

>_ WORKED EXAMPLE 4

# Clock arithmetic: time after N minutes

code/04_05_ClockArithmetic (excerpt)

```
const int MinutesPerDay = 24 * 60;             // 1440
int now = hour * 60 + minute;                  // minutes since midnight
int raw = now + delta;
int wrapped = ((raw % MinutesPerDay) + MinutesPerDay) % MinutesPerDay;

Console.WriteLine($"raw minutes = {raw}, raw % 1440 = {raw % MinutesPerDay}");
Console.WriteLine($"New time: {wrapped / 60:D2}:{wrapped % 60:D2}");
```

Run 1: forward

```
Current hour (0-23): 22
Current minute (0-59): 45
Minutes to add (negative = go back): 150
raw minutes = 1515, raw % 1440 = 75
New time: 01:15
```

Run 2: backward (negative delta)

```
Current hour (0-23): 0
Current minute (0-59): 20
Minutes to add (negative = go back): -45
raw minutes = -25, raw % 1440 = -25
New time: 23:35
```

**Plain % fails for −25** (gives −25); the safe modulo `((raw % 1440) + 1440) % 1440` gives 1415 → 23:35.

>_ WORKED EXAMPLE 5

# T.C. Kimlik No check digits (Week 1 algorithm in C#)

**code/04_10_KimlikCheckDigits (excerpt)**

```
int d1 = first9 / 100_000_000;
int d2 = first9 / 10_000_000 % 10;
// ... d3 to d8 in the same way ...
int d9 = first9 % 10;

int oddSum = d1 + d3 + d5 + d7 + d9;
int evenSum = d2 + d4 + d6 + d8;
int raw = 7 * oddSum - evenSum;
int d10 = ((raw % 10) + 10) % 10;  // safe modulo: result is 0..9
int d11 = (oddSum + evenSum + d10) % 10;
```

**Rules (Week 1)**

odd = d1 + d3 + d5 + d7 + d9

even = d2 + d4 + d6 + d8

d10 = (7·odd − even) mod 10

d11 = (d1 + … + d10) mod 10

**Needs the safe modulo!**

Console

```
First nine digits: 190909090
odd sum = 1, even sum = 36, 7*odd - even = -29
raw % 10 = -9  ->  d10 = 1, d11 = 8
Full number: 19090909018   (first digit non-zero: True)
```



PART 2

# Precedence, Assignment, ++ and --

Which operator runs first? · compound assignment · prefix vs postfix · string + number · overflow

# Operator precedence and associativity

| Level | Operators | Category | Order |
|---|---|---|---|
| 1 (first) | `x++  x--  f(x)  a.b` | primary / postfix | left → right |
| 2 | `+x  -x  !x  ++x  --x  (int)x` | unary / cast | right → left |
| 3 | `*  /  %` | multiplicative | left → right |
| 4 | `+  -` | additive | left → right |
| 5 | `<  >  <=  >=` | relational | left → right |
| 6 | `==  !=` | equality | left → right |
| 7 | && | logical AND | left → right |
| 8 | `\|\|` | logical OR | left → right |
| 9 | `?  :` | conditional | right → left |
| 10 (last) | `=  +=  -=  *=  /=  %=` | assignment | right → left |



### Same level?

Use the associativity:
10 - 4 - 3 = (10 - 4) - 3 = 3



### Unsure?

Add parentheses. They are always allowed and make intent clear.

# Evaluating an expression step by step

| Step | Expression | Next operation |
|---|---|---|
| 0 | `3 + 4 * 2 - 6 / 3 % 2` | 4 * 2 (leftmost of * / %) |
| 1 | `3 + 8 - 6 / 3 % 2` | 6 / 3 |
| 2 | `3 + 8 - 2 % 2` | 2 % 2 |
| 3 | `3 + 8 - 0` | 3 + 8 (leftmost of + -) |
| 4 | `11 - 0` | 11 - 0 |
| 5 | `11` | done |

```
Console.WriteLine(
  3 + 4 * 2 - 6 / 3 % 2);
```

```
Console
11
```

Precedence of % = precedence of * and /

**Two passes:** first every * / % from left to right, then every + - from left to right.

# The average of three grades — three versions

code/04_14_OperatorPitfalls (excerpt) — g1, g2, g3 are int

```
Console.WriteLine($"g1 + g2 + g3 / 3     = {g1 + g2 + g3 / 3}      (wrong: / before +)");
Console.WriteLine($"(g1 + g2 + g3) / 3   = {(g1 + g2 + g3) / 3}       (integer division)");
Console.WriteLine($"(g1 + g2 + g3) / 3.0 = {(g1 + g2 + g3) / 3.0:F2}    (correct)");
```

Console (grades 60, 70, 85)

```
Grade 3: 85
g1 + g2 + g3 / 3     = 158      (wrong: / before +)
(g1 + g2 + g3) / 3   = 71       (integer division)
(g1 + g2 + g3) / 3.0 = 71.67    (correct)
```

**All three compile.** The compiler checks syntax and types, not your intention. Test with numbers you can check by hand.

**158** only g3 is divided

**71** integer division

**71.67** divide by 3.0

# Compound assignment operators

| Operator | Example | Means | score |
|---|---|---|---|
| = | score = 10 | (store 10) | 10 |
| += | score += 5 | score = score + 5 | 15 |
| -= | score -= 3 | score = score - 3 | 12 |
| *= | score *= 2 | score = score * 2 | 24 |
| /= | score /= 5 | score = score / 5 | 4 |
| %= | score %= 3 | score = score % 3 | 1 |



The table runs top to bottom; each row starts from the previous score.

Console (04_13)

```
score += 5  ->  15
score -= 3  ->  12
score *= 2  ->  24
score /= 5  ->  4   (integer division)
score %= 3  ->  1
```



**Right side first:** `c *= 3 + 1;` means `c = c * (3 + 1);`

# Increment and decrement: prefix vs postfix

**code/04_13_IncrementDecrement (excerpt)**

```
int i = 5;
int a = i++;                        // a gets 5, then i becomes 6
Console.WriteLine($"a = i++  ->  a = {a}, i = {i}");

i = 5;
int b = ++i;                        // i becomes 6, then b gets 6
Console.WriteLine($"b = ++i  ->  b = {b}, i = {i}");

int j = 3;
int c = j-- * 10;                   // uses 3, then j becomes 2
Console.WriteLine($"c = j-- * 10  ->  c = {c}, j = {j}");
```

Console

```
a = i++  ->  a = 5, i = 6
b = ++i  ->  b = 6, i = 6
c = j-- * 10  ->  c = 30, j = 2
```

**i++** → use old value, then add 1

**++i** → add 1, then use new value

| a = i++; | i | a |
|---|---|---|
| start | 5 | ? |
| a ← old i | 5 | 5 |
| i ← i + 1 | 6 | 5 |

| b = ++i; | i | b |
|---|---|---|
| start | 5 | ? |
| i ← i + 1 | 6 | 6 |
| b ← new i | 6 | 6 |

# String + number and integer overflow

**string + number  (g1 = 60, g2 = 70)**

```
Console.WriteLine("Sum: " + g1 + g2);       // text first -> joins as text
Console.WriteLine("Sum: " + (g1 + g2));     // parentheses -> adds first
Console.WriteLine(g1 + g2 + " is the sum"); // left to right: adds, then joins
```

Console

```
Sum: 6070
Sum: 130
130 is the sum
```

**overflow**

```
int big = int.MaxValue;              // 2147483647
Console.WriteLine(big + 1);          // wraps around
Console.WriteLine((long)big + 1);    // long: correct
// Console.WriteLine(checked(big + 1));  -> OverflowException
```

Console

```
-2147483648
2147483648
```



**Safer habits:** use interpolation `$"Sum: {g1 + g2}"` instead of + for output; use `long` when values can exceed about 2.1 billion.



PART 3

# The Math Class

Pow · Sqrt · Abs · Round · Floor · Ceiling · Max · Min · PI · worked examples

# The Math toolbox

| Method | Meaning | Example | Prints |
|---|---|---|---|
| `Math.Pow(x, y)` | x to the power y | `Math.Pow(2, 10)` | `1024` |
| `Math.Sqrt(x)` | square root | `Math.Sqrt(16)` | `4` |
| `Math.Abs(x)` | absolute value \|x\| | `Math.Abs(-7.5)` | `7.5` |
| `Math.Max(a, b)` | the larger | `Math.Max(3, 7)` | `7` |
| `Math.Min(a, b)` | the smaller | `Math.Min(3, 7)` | `3` |
| `Math.Round(x)` | nearest (ties → even) | `Math.Round(2.5)` | `2` |
| `Math.Floor(x)` | round down | `Math.Floor(-2.5)` | `-3` |
| `Math.Ceiling(x)` | round up | `Math.Ceiling(2.1)` | `3` |
| `Math.PI` | the constant π | `Math.PI` | `3.141592653589793` |



### Returns double

int k = Math.Pow(2, 3);
→ error CS0266
Fix: (int)Math.Pow(2, 3)



### Squares

x * x is simpler and exact; use Pow for $(1 + r)^n$.

# Rounding: Math.Round uses banker's rounding

| x | Round(x) | Round(x, AwayFromZero) | Floor(x) | Ceiling(x) | (int)x |
|---|---|---|---|---|---|
| 2.5 | 2 | 3 | 2 | 3 | 2 |
| 3.5 | 4 | 4 | 3 | 4 | 3 |
| -2.5 | -2 | -3 | -3 | -2 | -2 |
| -2.7 | -3 | -3 | -3 | -2 | -2 |



### Banker's rounding

Ties go to the even neighbour: 0.5→0, 1.5→2, 2.5→2.



### School rounding

Math.Round(x, MidpointRounding.AwayFromZero)



### Floor / Ceiling / cast

Down / up / toward zero. They differ for negatives!

>_ WORKED EXAMPLE 6

# Quadratic equation: discriminant and roots

**code/04_07_QuadraticRoots (excerpt) — a, b, c are double**

```
double delta = b * b - 4 * a * c;
double x1 = (-b + Math.Sqrt(delta)) / (2 * a);    // parentheses are essential
double x2 = (-b - Math.Sqrt(delta)) / (2 * a);

Console.WriteLine($"delta = {delta}");
Console.WriteLine(delta >= 0 ? "Real roots exist." : "No real roots (Sqrt of a negative is NaN).");
```

$x^2 - 3x + 2 = 0$

```
a = 1
b = -3
c = 2
delta = 1
Real roots exist.
x1 = 2.0000
x2 = 1.0000
```

$x^2 + 2x + 5 = 0$

```
a = 1
b = 2
c = 5
delta = -16
No real roots (Sqrt of a negative is NaN).
x1 = NaN
x2 = NaN
```

**Wrong:**

```
-b + Math.Sqrt(delta)
  / 2 * a
```

divides only the root, then MULTIPLIES by a.

>_ WORKED EXAMPLE 7

# Compound interest  A = P(1 + r/n)^(nt)

$$A = P \cdot (1 + r / n)^{n \cdot t}$$

| Symbol | Meaning | Run |
|---|---|---|
| P | principal | 10000 TL |
| r | annual rate | 40 % → 0.40 |
| n | periods / year | 12 |
| t | years | 2 |

**int trap:** `40 / 100` is 0! Read the rate as double.

code/04_08_CompoundInterest (excerpt)

```
double r = ratePercent / 100;
double amount = principal * Math.Pow(1 + r / n, n * t);
double simple = principal * (1 + r * t);
```

Console

```
Principal P (TL): 10000
Annual interest rate (%): 40
Compounding periods per year n: 12
Years t: 2
Compound amount : 21966.99 TL
Interest earned : 11966.99 TL
Simple interest : 18000.00 TL
```

Check: 1000 TL, 10 %, n = 1, t = 3 → 1331.00



PART 4

# Relational and Logical Operators

== != < > <= >= → bool · && || ! · short-circuit · De Morgan · ?:

# Relational operators produce a bool

| C# | Pseudocode | Meaning | Example | Prints |
|---|---|---|---|---|
| `==` | = | equal to | `5 == 5` | `True` |
| `!=` | ≠ | not equal to | `5 != 3` | `True` |
| `<` | < | less than | `3 < 2` | `False` |
| `>` | > | greater than | `7 > 2` | `True` |
| `<=` | ≤ | less than or equal | `5 <= 5` | `True` |
| `>=` | ≥ | greater than or equal | `4 >= 9` | `False` |

```
int grade = 72;
bool passed = grade >= 50;
Console.WriteLine(passed);
```

Console

```
True
```

pseudocode ← is C# =
pseudocode = is C# ==

**= assigns, == compares.** `bool same = (x = 5);` → error CS0029: Cannot implicitly convert type 'int' to 'bool'.

# Why 0.1 + 0.2 == 0.3 is False

```
double sum = 0.1 + 0.2;
Console.WriteLine(sum);
Console.WriteLine(sum == 0.3);
Console.WriteLine(Math.Abs(sum - 0.3) < 1e-9);
Console.WriteLine(0.1m + 0.2m == 0.3m);
```

Console

```
0.30000000000000004
False
True
True
```

### Do not

compare doubles with ==,
e.g. sum == 0.3

### Do

Math.Abs(a - b) < 1e-9
(a tolerance, "epsilon").

### Money

Use decimal (suffix m): base-10 digits are stored exactly.

# Logical operators and truth tables

| p | q | p && q | p \|\| q | !p |
|---|---|---|---|---|
| False | False | False | False | True |
| False | True | False | True | True |
| True | False | False | True | False |
| True | True | True | True | False |

**&&** — **AND · ve**
both must be true

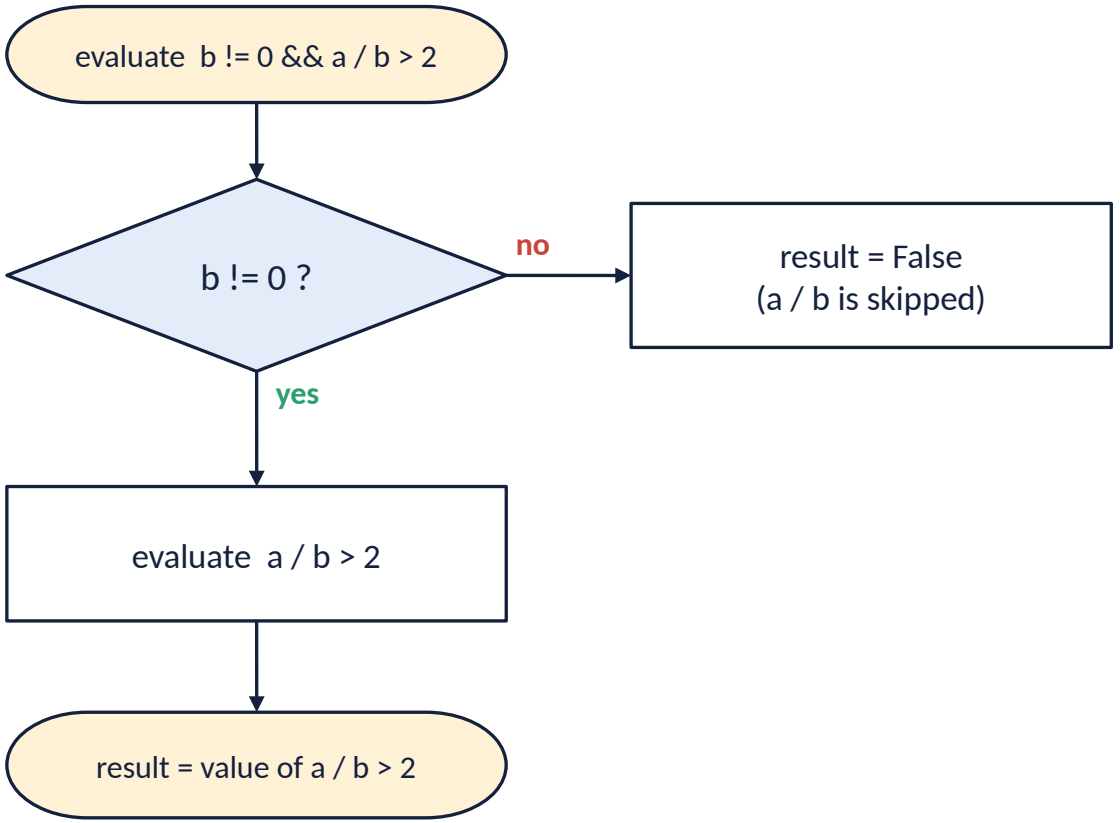
**||** — **OR · veya**
at least one true

**!** — **NOT · değil**
flips true ↔ false



**Precedence:** ! before && before ||. So `a || b && c` means `a || (b && c)`. When you mix && and ||, write the parentheses yourself.

# Short-circuit evaluation: a built-in guard




```
int a = 10, b = 0;
bool bigRatio = b != 0 && a / b > 2;
Console.WriteLine(
  $"b != 0 && a / b > 2  ->  {bigRatio}");
```

```
Console
b != 0 && a / b > 2  ->  False
```

**`false &&` …** → false, right side skipped

**`true ||` …** → true, right side skipped

**Order matters:** `a / b > 2 && b != 0` crashes when b is 0.

# De Morgan's laws and range tests

**De Morgan's laws**

```
!(A && B)   ==   !A || !B
!(A || B)   ==   !A && !B
```

negate each part and swap && ↔ ||

**Example**

"not between 18 and 65":

```
!(age >= 18 && age <= 65)
age < 18 || age > 65
```

(the same thing)

## Range test $0 \le x \le 100$




```
x >= 0 && x <= 100      // correct
```

**`0 <= x <= 100`**

error CS0019: Operator '<=' cannot be applied to operands of type 'bool' and 'int'

(0 <= x) is already a bool.

>_ WORKED EXAMPLE 8

# Leap year as one boolean expression (+ the ?: operator)

code/04_06_LeapYear (excerpt)

```
bool divisibleBy4 = year % 4 == 0;
bool divisibleBy100 = year % 100 == 0;
bool divisibleBy400 = year % 400 == 0;
bool isLeap = (divisibleBy4 && !divisibleBy100) || divisibleBy400;

Console.WriteLine($"divisible by 4: {divisibleBy4}, by 100: {divisibleBy100}, by 400: {divisibleBy400}");
Console.WriteLine($"isLeap = {isLeap}");
Console.WriteLine($"{year} is {(isLeap ? "a leap year (366 days)" : "not a leap year (365 days)")}");
```

| year | % 4 == 0 | % 100 == 0 | % 400 == 0 | isLeap |
|---|---|---|---|---|
| 2024 | True | False | False | **True** |
| 1900 | True | True | False | **False** |
| 2000 | True | True | True | **True** |
| 2026 | False | False | False | **False** |

**Conditional operator ?:**

`condition ? valueIfTrue : valueIfFalse`

`isLeap ? "a leap year" : "not a leap year"`

Only for choosing a value. Full if-else: Week 5.

>_ COMMON MISTAKES

# Six operator bugs to avoid

### Integer division

```
✗ 7 / 2 expected 3.5
✓ write 7 / 2.0 or (double)a / b
```

### Missing parentheses

```
✗ g1 + g2 + g3 / 3
✓ (g1 + g2 + g3) / 3.0
```

### Chained comparison

```
✗ 0 <= x <= 100
✓ x >= 0 && x <= 100
```

### = instead of ==

```
✗ bool ok = (x = 5);
✓ bool ok = (x == 5);
```

### == on doubles

```
✗ 0.1 + 0.2 == 0.3
✓ Math.Abs(a - b) < 1e-9
```

### Negative %

```
✗ -7 % 3 is -1
✓ ((x % m) + m) % m
```

# Quiz 1: Arithmetic

| # | Console.WriteLine( … ); | Output? |
|---|---|---|
| 1 | `17 / 5` | |
| 2 | `17 % 5` | |
| 3 | `17 / 5.0` | |
| 4 | `(double)(17 / 5)` | |
| 5 | `-17 % 5` | |
| 6 | `5 % 17` | |
| 7 | `1 / 2 * 10.0` | |
| 8 | `-9 / 2` | |



**Write exactly what is printed.**

bool → True / False

3.0 prints as 3

Work in pairs · 5 min

# Quiz 1 answers: Arithmetic

| # | Expression | Output | Why |
|---|---|---|---|
| 1 | `17 / 5` | **3** | int / int: the fraction is dropped |
| 2 | `17 % 5` | **2** | 17 = 3 · 5 + 2 |
| 3 | `17 / 5.0` | **3.4** | one double operand → real division |
| 4 | `(double)(17 / 5)` | **3** | int division first; the cast comes too late |
| 5 | `-17 % 5` | **-2** | the sign of % follows the dividend (−17) |
| 6 | `5 % 17` | **5** | 5 = 0 · 17 + 5 |
| 7 | `1 / 2 * 10.0` | **0** | 1 / 2 is the int 0; 0 * 10.0 = 0 |
| 8 | `-9 / 2` | **-4** | truncation toward zero (not −5) |

# Quiz 2: Precedence and Strings

| # | Console.WriteLine( … ); | Output? |
|---|---|---|
| 1 | `2 + 3 * 4` | |
| 2 | `10 - 4 - 3` | |
| 3 | `100 / 10 / 5` | |
| 4 | `20 / 3 * 3` | |
| 5 | `2 * 3 % 4` | |
| 6 | `1 + 2 + "3"` | |
| 7 | `"1" + 2 + 3` | |
| 8 | `'A' + 1` | |



**Write exactly what is printed.**

bool → True / False

3.0 prints as 3

Work in pairs · 5 min

# Quiz 2 answers: Precedence and Strings

| # | Expression | Output | Why |
|---|---|---|---|
| 1 | `2 + 3 * 4` | **14** | * before + |
| 2 | `10 - 4 - 3` | **3** | left to right: (10 − 4) − 3 |
| 3 | `100 / 10 / 5` | **2** | left to right: (100 / 10) / 5 |
| 4 | `20 / 3 * 3` | **18** | 20 / 3 = 6 (int), then 6 * 3 |
| 5 | `2 * 3 % 4` | **2** | same level, left to right: 6 % 4 |
| 6 | `1 + 2 + "3"` | **33** | 1 + 2 = 3 first, then "3" + "3" |
| 7 | `"1" + 2 + 3` | **123** | text first → every + joins text |
| 8 | `'A' + 1` | **66** | char 'A' has code 65; char + int → int |

# Quiz 3: ++, -- and compound assignment

**Snippet A**

```
int i = 5;
int a = i++ + 2;
Console.WriteLine($"{a} {i}");
```

**Snippet B**

```
int i = 5;
int a = ++i + 2;
Console.WriteLine($"{a} {i}");
```

**Snippet C**

```
int n = 7;
Console.WriteLine(n++);
Console.WriteLine(n);
Console.WriteLine(--n);
```

**Snippet D**

```
int k = 10;
k -= 3;
k *= 2;
k %= 5;
Console.WriteLine(k);
```

**Snippet E**

```
int m = 10;
m /= 4;
double d = 10;
d /= 4;
Console.WriteLine($"{m} {d}");
```

**Snippet F**

```
int c = 2;
c *= 3 + 1;
Console.WriteLine(c);
```

# Quiz 3 answers: ++, -- and compound assignment

**Snippet A**

```
int i = 5;
int a = i++ + 2;
Console.WriteLine($"{a} {i}");
```

**7 6** i++ gives the old value 5

**Snippet B**

```
int i = 5;
int a = ++i + 2;
Console.WriteLine($"{a} {i}");
```

**8 6** ++i gives the new value 6

**Snippet C**

```
int n = 7;
Console.WriteLine(n++);
Console.WriteLine(n);
Console.WriteLine(--n);
```

**7 | 8 | 7** print 7, then n = 8; --n makes 7

**Snippet D**

```
int k = 10;
k -= 3;
k *= 2;
k %= 5;
Console.WriteLine(k);
```

**4** 10 → 7 → 14 → 14 % 5 = 4

**Snippet E**

```
int m = 10;
m /= 4;
double d = 10;
d /= 4;
Console.WriteLine($"{m} {d}");
```

**2 2.5** int /= keeps int; double /= is real

**Snippet F**

```
int c = 2;
c *= 3 + 1;
Console.WriteLine(c);
```

**8** c = c * (3 + 1): right side first

# Quiz 4: Relational and Logical

| # | Console.WriteLine( ... ); | Output? |
|---|---|---|
| 1 | `5 > 3 && 2 > 4` | |
| 2 | `7 % 2 == 1` | |
| 3 | `10 / 4 == 2.5` | |
| 4 | `0.1 + 0.2 == 0.3` | |
| 5 | `true \|\| false && false` | |
| 6 | `!true \|\| true` | |
| 7 | `b != 0 && 10 / b > 1` | |
| 8 | `b == 0 \|\| 10 / b > 1` | |



**Write exactly what is printed.**

bool → True / False

3.0 prints as 3

Work in pairs · 5 min

```
int b = 0;
```

# Quiz 4 answers: Relational and Logical

| # | Expression | Output | Why |
|---|---|---|---|
| 1 | `5 > 3 && 2 > 4` | **False** | true && false |
| 2 | `7 % 2 == 1` | **True** | % before ==; 1 == 1 |
| 3 | `10 / 4 == 2.5` | **False** | 10 / 4 is the int 2 |
| 4 | `0.1 + 0.2 == 0.3` | **False** | sum is 0.30000000000000004 |
| 5 | `true \|\| false && false` | **True** | && first: true \|\| (false && false) |
| 6 | `!true \|\| true` | **True** | ! binds tightest: false \|\| true |
| 7 | `b != 0 && 10 / b > 1` | **False** | left side false → right side skipped |
| 8 | `b == 0 \|\| 10 / b > 1` | **True** | left side true → right side skipped |

Setup: `int b = 0;`

# Quiz 5: The Math Class

| # | Console.WriteLine( … ); | Output? |
|---|---|---|
| 1 | `Math.Round(2.5)` | |
| 2 | `Math.Round(3.5)` | |
| 3 | `Math.Round(-2.5)` | |
| 4 | `Math.Floor(-2.5)` | |
| 5 | `Math.Ceiling(-2.5)` | |
| 6 | `(int)-2.7` | |
| 7 | `Math.Pow(2, 10)` | |
| 8 | `Math.Max(3, Math.Min(10, 7))` | |



**Write exactly what is printed.**

bool → True / False

3.0 prints as 3

Work in pairs · 5 min

# Quiz 5 answers: The Math Class

| # | Expression | Output | Why |
|---|---|---|---|
| 1 | `Math.Round(2.5)` | **2** | banker's rounding: tie → even |
| 2 | `Math.Round(3.5)` | **4** | tie → even neighbour 4 |
| 3 | `Math.Round(-2.5)` | **-2** | tie → even neighbour −2 |
| 4 | `Math.Floor(-2.5)` | **-3** | Floor goes down (toward −∞) |
| 5 | `Math.Ceiling(-2.5)` | **-2** | Ceiling goes up (toward +∞) |
| 6 | `(int)-2.7` | **-2** | a cast truncates toward zero |
| 7 | `Math.Pow(2, 10)` | **1024** | $2^{10}$ |
| 8 | `Math.Max(3, Math.Min(10, 7))` | **7** | inner first: Min = 7, Max(3, 7) = 7 |

>_ PRACTICE

# In-class exercises

## E1 · Coins

Read an amount in kuruş (e.g. 385). Print how many 1 TL, 50, 25, 10, 5 and 1 kuruş coins give it (fewest coins). Use / and %.

## E2 · Four digits

Read a four-digit number. Print its digits, their sum and the number reversed (e.g. 2026 → 6202).

## E3 · Heron

Read the sides a, b, c of a triangle. Print s = (a + b + c) / 2 and the area √(s(s−a)(s−b)(s−c)) with 2 decimals.

## E4 · Checks

Read an int x. Print bools: isEven, isPositive, isTwoDigit (10 ≤ |x| ≤ 99), isMultipleOf3Or5.

# Homework (due next week)

| | | |
|---|---|---|
| H1 | **Kimlik validator** | Read an 11-digit number as long. Compute d10 and d11 from the first nine digits and print one bool: is the number valid? (Use made-up numbers only.) |
| H2 | **Time difference** | Read two clock times (hh mm, hh mm). Print the minutes from the first to the second, wrapping past midnight (22:30 → 01:15 = 165). |
| H3 | **Loan payment** | Monthly payment M = P·i(1+i)$^n$ / ((1+i)$^n$ − 1), i = annual rate / 12, n = months. Print M and the total paid. |
| H4 | **Predict sheet** | Solve the 12 predict-the-output items in the notes on paper first, then check them by running C#. |

>_ SUMMARY

# Key Takeaways

01 int / int is integer division; make one operand double for a real result.

02 % gives the remainder; its sign follows the dividend. Safe modulo: ((x % m) + m) % m.

03 Precedence: unary → * / % → + - → relational → == != → && → || → ?: → =. Use parentheses.

04 i++ gives the old value, ++i the new one; x op= y means x = x op (y).

05 Math: Pow/Sqrt return double; Round uses banker's rounding by default.

06 Relational → bool; never == on doubles; && || short-circuit; 0 <= x <= 100 is an error.

NEXT WEEK

## Decisions: if-else, else-if and switch



MCMP113
Course materials